

《计算概论》课程 程序设计部分

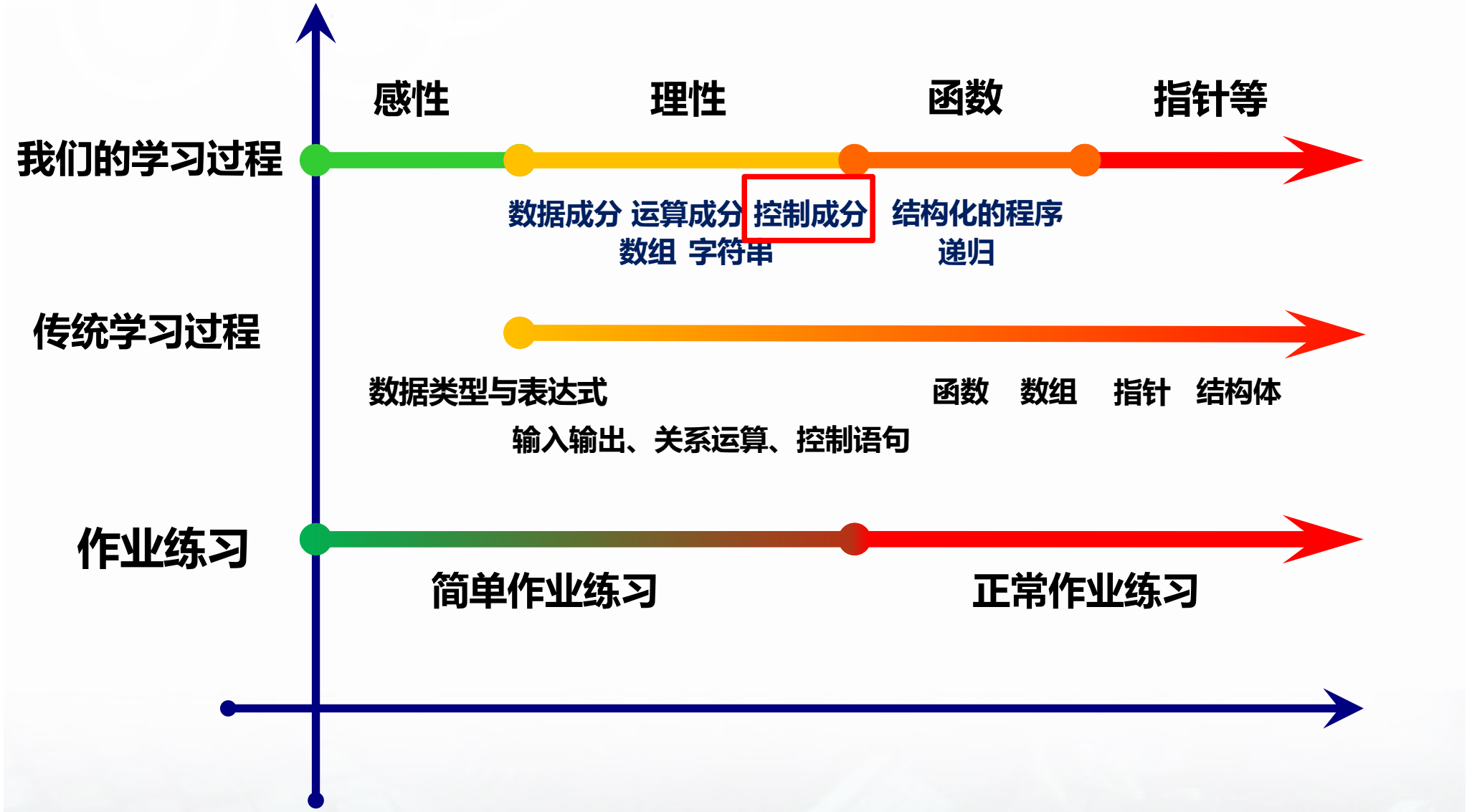
C/C++ 语言的基本成分 — 控制成分

董 豪

北京大学 计算机学院 前沿计算研究中心

hao.dong@pku.edu.cn

再谈，我们的进度安排


感性
理性
函数
指针等
我们的学习过程
数据成分 运算成分 控制成分 结构化的程序
数组 字符串
递归
传统学习过程
数据类型与表达式
函数 数组 指针 结构体
输入输出、关系运算、控制语句
作业练习
简单作业练习
正常作业练习

问题1：

是不是 无论我们在程序里写什么“单词”，计算机都能明白？

问题2：

是不是 无论我们在程序里写什么“数”和“计算符号”，计算机都能明白？

问题3：

世界上可能要用“程序来表达的逻辑”纷繁复杂，程序设计语言里面得有多少种“句式”才能够用？

问题3：

世界上可能要用“程序来表达的逻辑”纷繁复杂，程序设计语言里面得有多少种“句式”才能够用?

- **不多！三种而已！**

- C. Bohm & G. Jacopini, "Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules," Communications of the ACM, vol9(5) May 1966, pp 366-371.

程序设计语言的构成

- **语言的种类千差万别，但是，一般说来，基本成分不外四种：**
  - **数据成分**，用以描述程序中所涉及的数据；
  - **运算成分**，用以描述程序中所包含的运算；
  - **控制成分**，用以表达程序中的控制构造；
  - **传输成分**，用以表达程序中数据的传输；

——计算机科学技术百科全书

计算机程序的基本结构

- 什么样的结构才能支持程序运行的逻辑?
  - 1966年，G. Jacopini 和 C. Bohm在“Communications of the ACM”上发表论论文“Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules” 。
- 从理论上证明了：任何具有**单入口单出口**的程序都可以用三种基本结构表达：
  - 顺序结构
  - 分支结构 **if else** **switch case**
  - 循环结构 **for** **while** **do..while**

  break **continue**

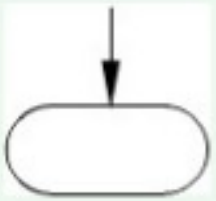
  goto
- C. Bohm & G. Jacopini, "Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules," Communications of the ACM, vol9(5) May 1966, pp 366-371.

程序流程图

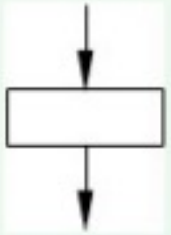
（1）起始框



（2）终止框

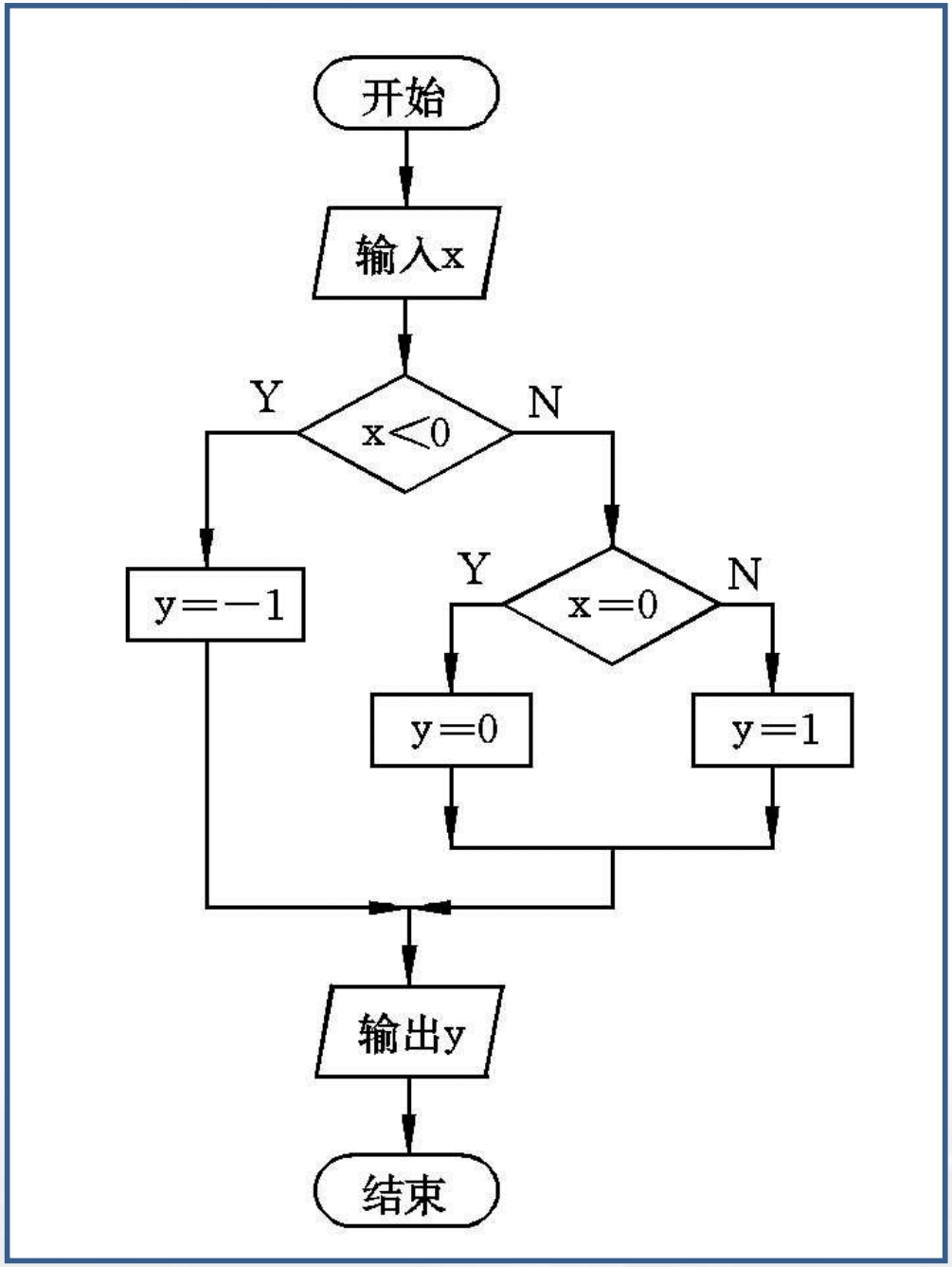


（3）执行框

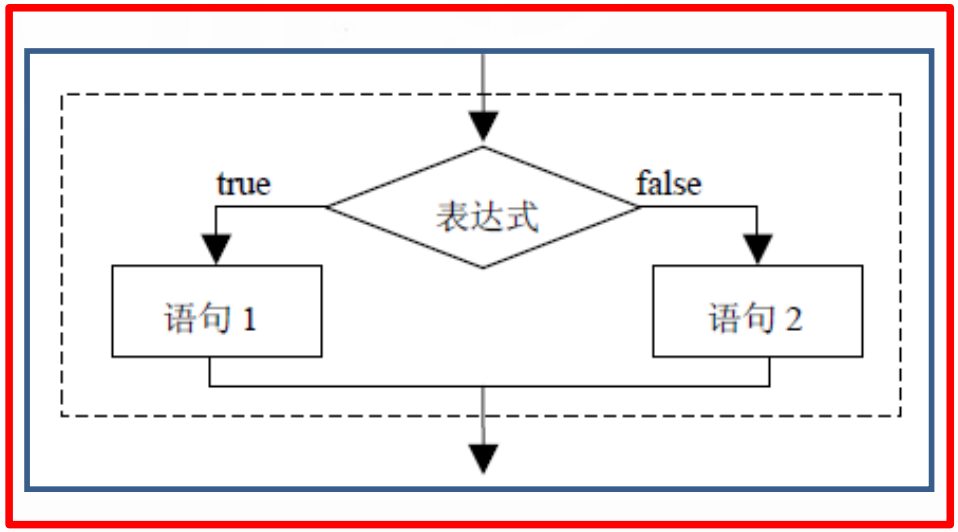


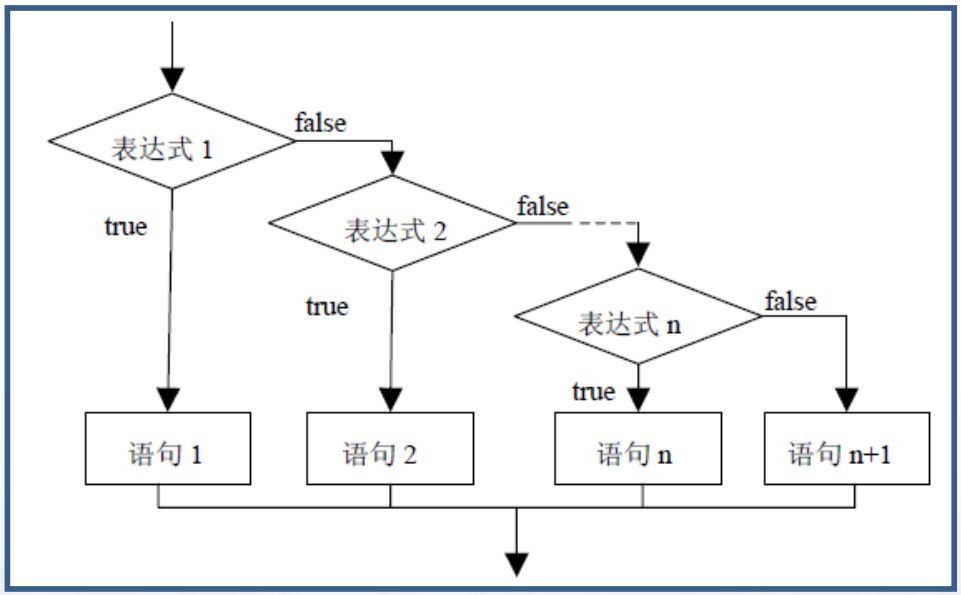
（4）判别框

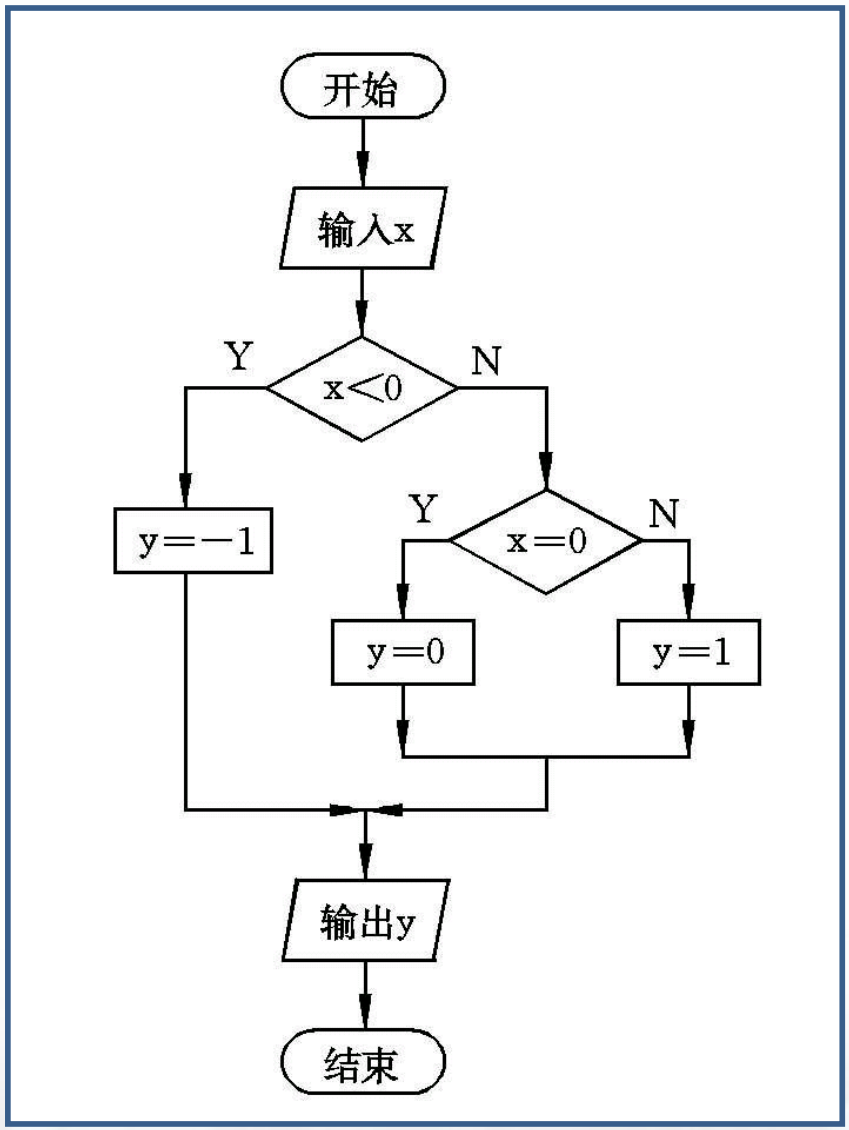




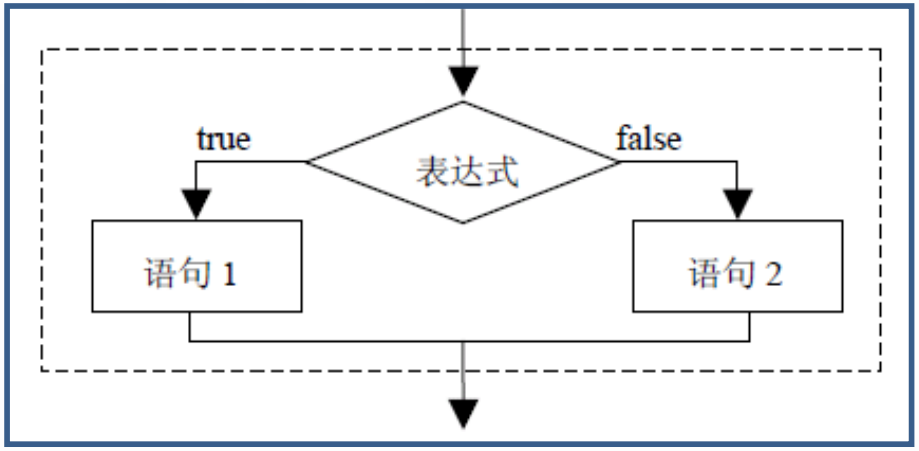
再谈 分支语句

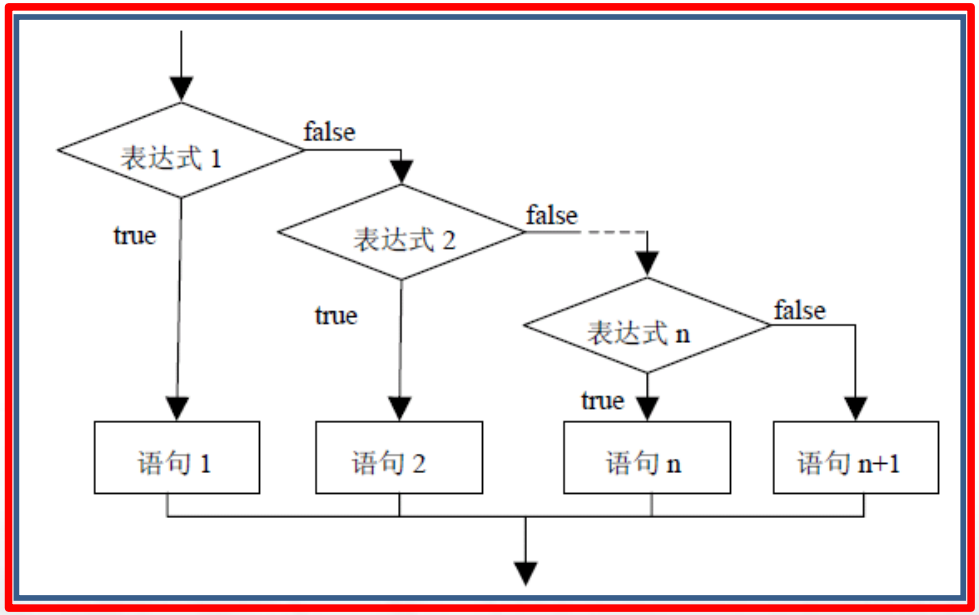

true
表达式
false
语句 1
语句 2

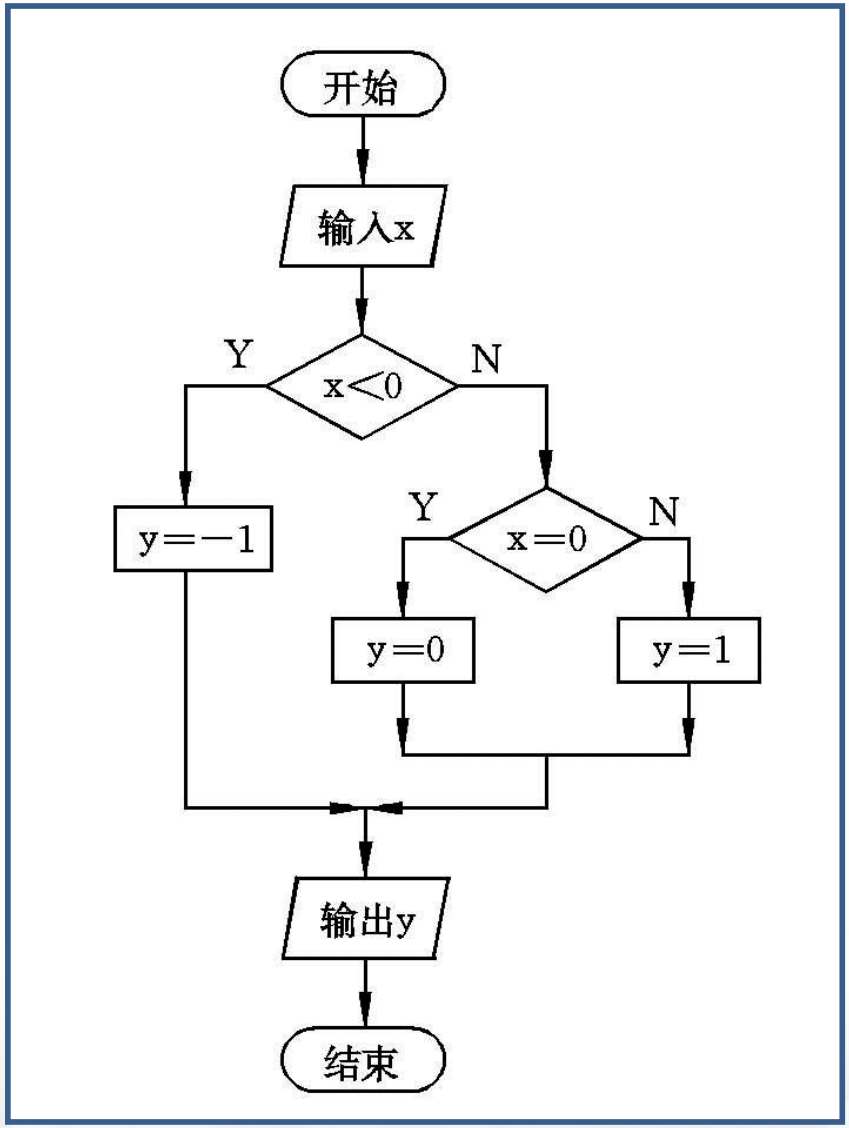

表达式 1
false
true
表达式 2
false
true
表达式 n
false
true
语句 1
语句 2
语句 n
语句 n+1


开始
输入x
Y
x<0
N
y=−1
Y
x=0
N
y=0
y=1
输出y
结束

再谈 分支语句


true
表达式
false
语句 1
语句 2


表达式 1
false
true
表达式 2
false
true
表达式 n
false
true
语句 1
语句 2
语句 n
语句 n+1


开始
输入x
Y
x<0
N
y=−1
Y
x=0
N
y=0
y=1
输出y
结束

```
#include<iostream>
using namespace std;
int main()
{
        float  weight = 0, height = 0, healthRate = 0;
        cin >> weight >> height;
        healthRate = weight / (height*height);
        if ((18 <= healthRate) && (healthRate <= 25))
                cout << "体重适中！ " << endl;
        else if ((25 < healthRate) && (healthRate <= 30))
                cout << "超重！ 注意控制！ " << endl;
        else if (( 30 < healthRate) && (healthRate <= 35))
                cout << "肥胖！ 减肥吧！ " << endl;
        else if ((35 < healthRate) && (healthRate <= 40))
                cout << "重度肥胖！ 别吃了！ " << endl;
        else
                cout << "请直接拨打120！ " << endl;
        return 0;
}
```

```cpp
#include <iostream>
using namespace std;
int main() {
    int a=3;
    if(0<a<5)
        cout << "0<a<5" << endl;
    if(0<a && a<5)
        cout << "0<a && a<5" << endl;
    return 0;
}
```

0<a<5

0<a && a<5

C++里面，if(0<a<5) 什么意思

分享

在 C++ 中，`if(0 < a < 5)` 并不是你可能期望的 “判断 `a` 是否大于 0 且小于 5” 的意思，这是一个常见的逻辑误区。

这个表达式的执行顺序是从左到右逐步计算的：

1. 首先计算 `0 < a`，这会得到一个布尔结果（`true` 或 `false`）
2. 在 C++ 中，布尔值会被隐式转换为整数：`true` 转换为 1，`false` 转换为 0
3. 然后用这个转换后的结果（0 或 1）与 5 进行比较

例如：

- 当 `a = 3` 时：`0 < 3` 为 `true`（即 1），然后 `1 < 5` 也为 `true`，所以条件成立
- 当 `a = 6` 时：`0 < 6` 为 `true`（即 1），然后 `1 < 5` 也为 `true`，条件成立（这显然不符合预期）
- 当 `a = -1` 时：`0 < -1` 为 `false`（即 0），然后 `0 < 5` 为 `true`，条件成立（这也不符合预期）

要正确判断 `a` 是否在 0 到 5 之间，应该使用逻辑与运算符 `&&`：

cpp

运行

```cpp
if (0 < a && a < 5) {
    // 当a大于0且小于5时执行
}
```

再谈 分支结构

```
if (x > y)
        ...
```

```
if (x > y)
        ...
else
        ...
```

```
if (x > y)
        ...
else if (x < y)
        ...
```

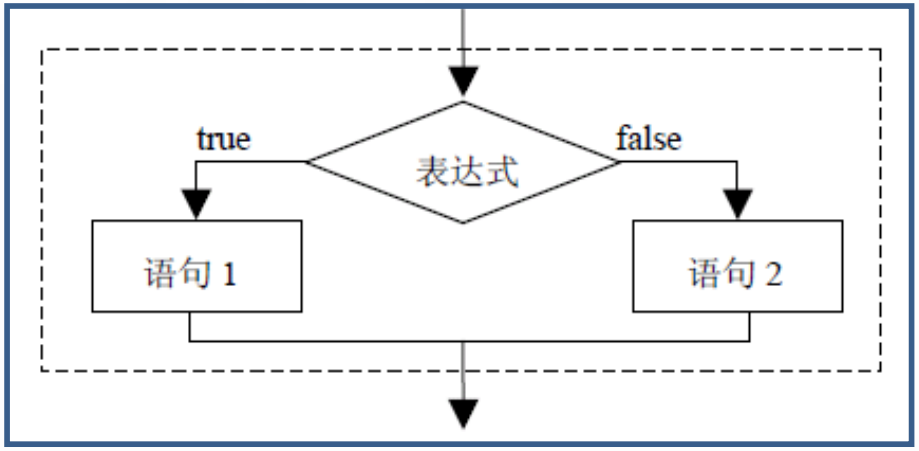
```
if (x > y)
        ...
else
        ...
el
```

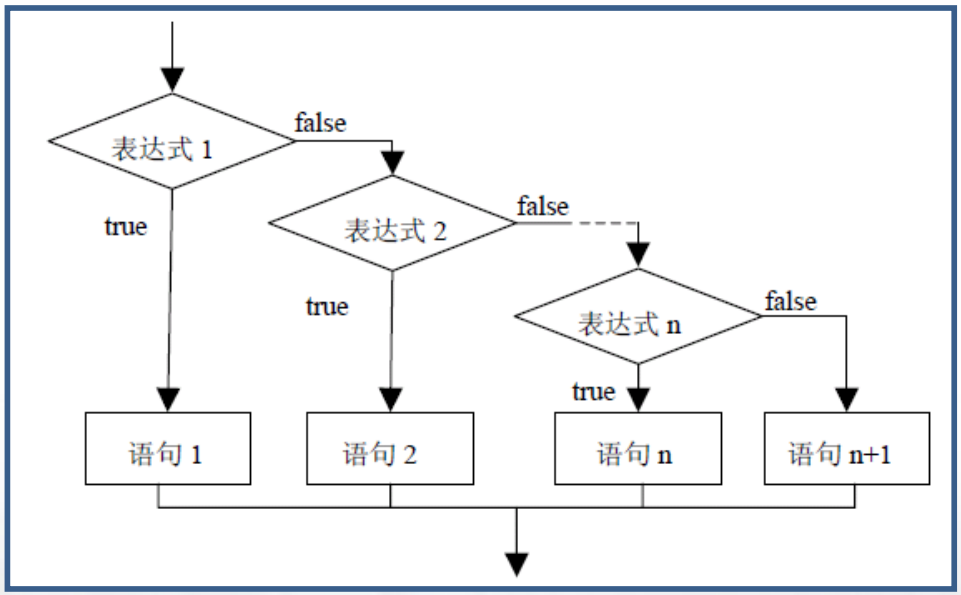
```
if (x > y)
        ...
else if (x < y)
        ...
else if (x == y)
        ...
```

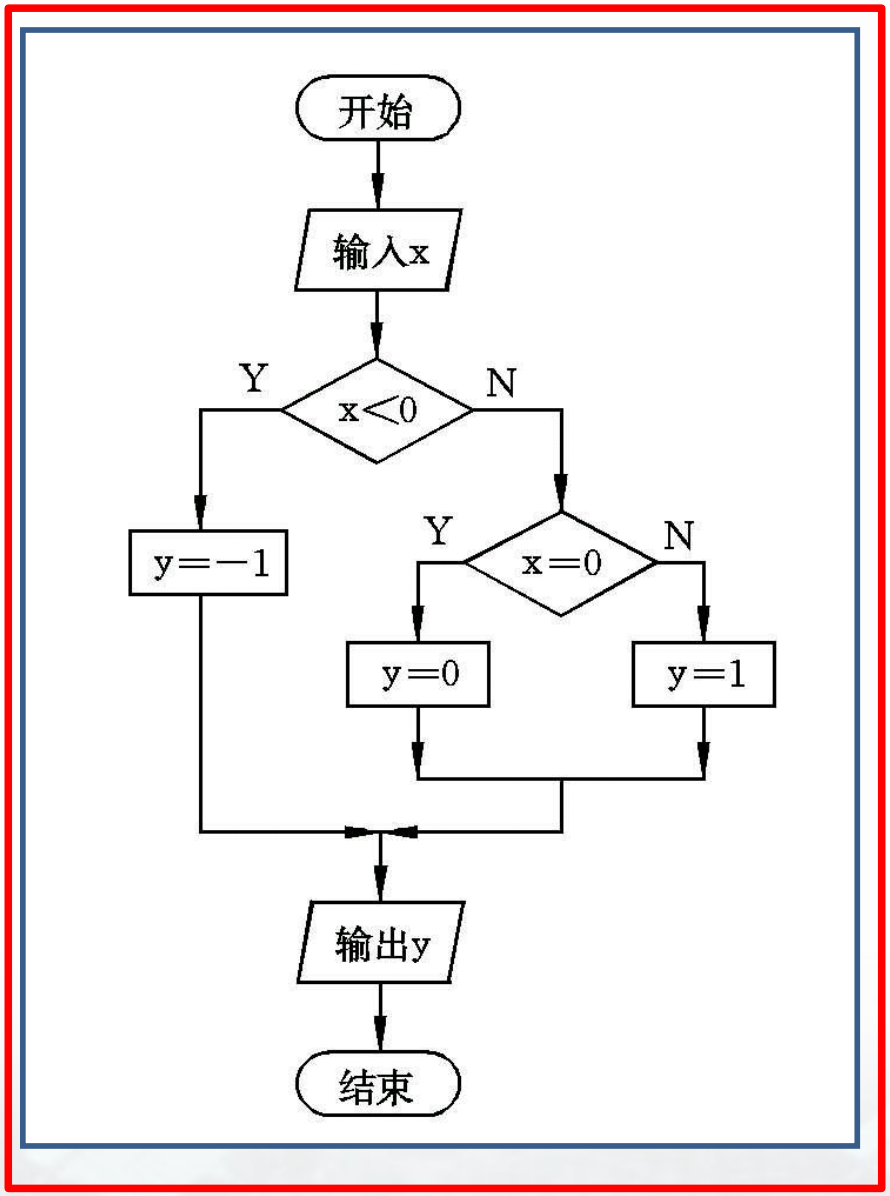
```
if (x > y)
        ...
else if (x < y)
        ...
else
        ...
```

```
main.cpp: In function ‘int main()’:
main.cpp:19:9: error: ‘else’ without a previous ‘if’
   19 |         else if (x < y)
      |         ^~~~
```

分支语句


true
表达式
false
语句 1
语句 2


表达式 1
false
true
表达式 2
false
true
表达式 n
false
true
语句 1
语句 2
语句 n
语句 n+1


开始
输入x
Y
x<0
N
y=−1
Y
x=0
N
y=0
y=1
输出y
结束

```
#include <iostream>
using namespace std;
int main()
{
        int year = 0;
        cin >> year;
        if (year % 4 == 0)
        {
                if (year % 100 == 0)
                {
                        if (year % 400 == 0)
                                cout << "Y";
                        else
                                cout << "N";
                }
                else
                        cout << "Y";
        }
        else
                cout << "N";
        return 0;
}
```

1

2

2

1



if 语句使用须知

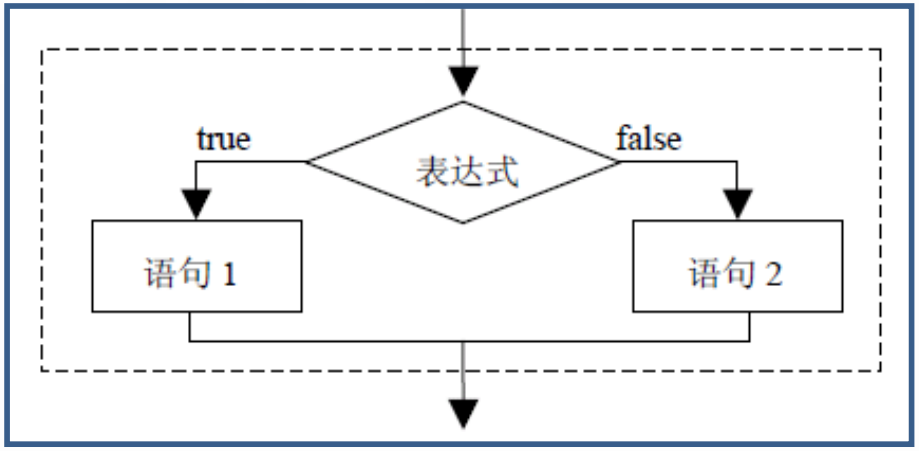
- **用法：**

  if (*条件表达式*) *语句*

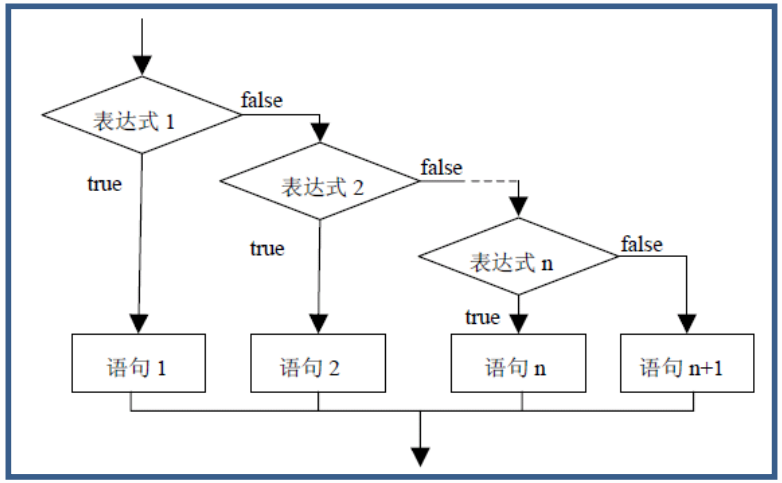
- **在执行 if 语句前先对 *条件表达式* 求解**
  - ***条件表达式*** **的值期望一个布尔类型，但它可以由任意的类型隐式转换得来。**

```
if ('a') cout << 'a' << endl;
if (3) cout << "OK" << endl;
```

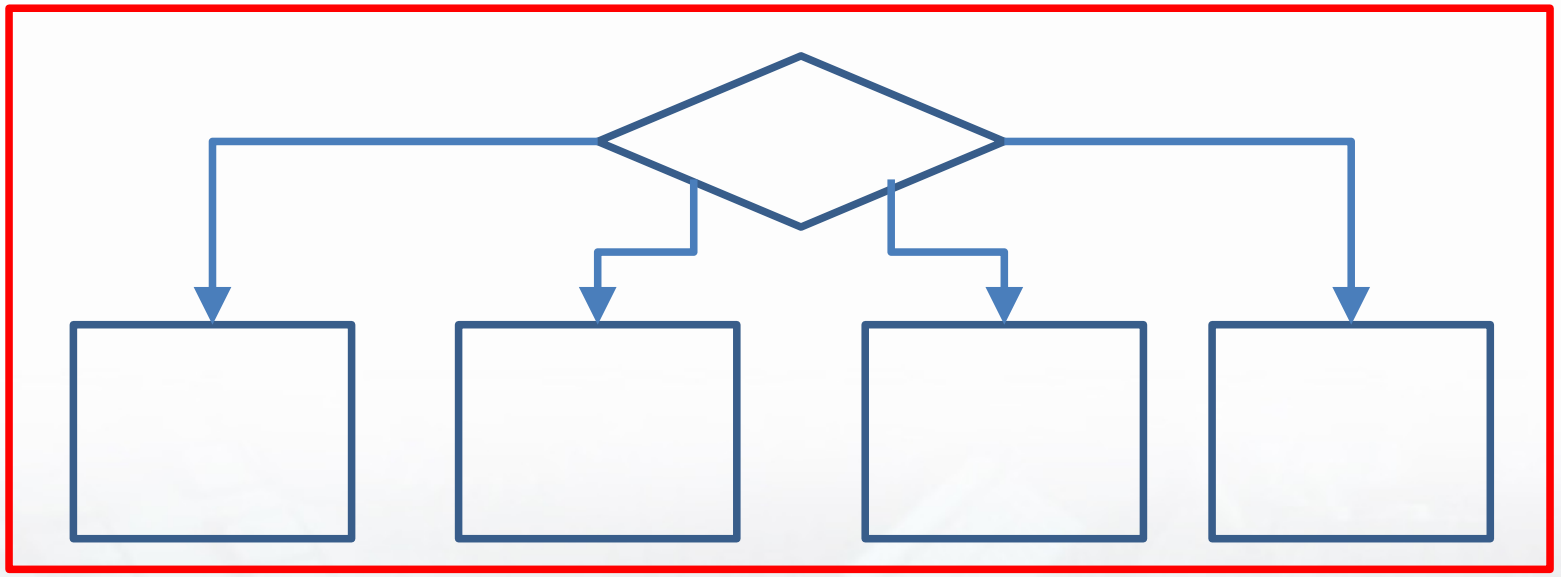
再谈 分支语句



if … else …



if … else if … else if … else



如何实现多分支?

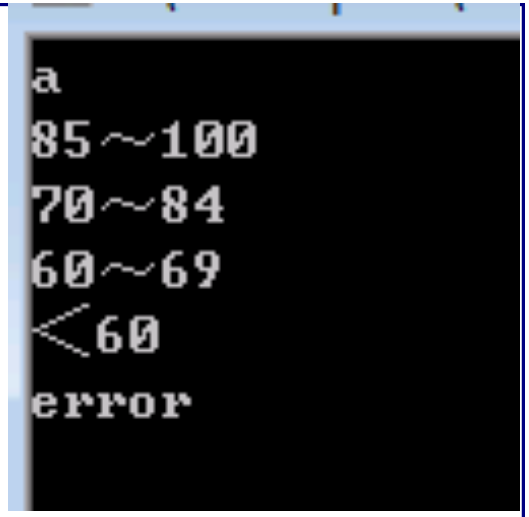
多分支语句

- switch 语句的一般形式：

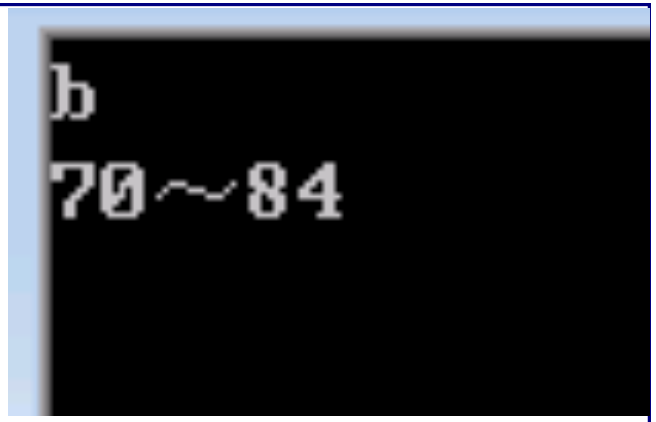
```
switch (表达式) {
        case 常量表达式1:
             语句1
        case 常量表达式2:
             语句2
        case 常量表达式n:
             语句n
        default:
             语句n+1
}
```

- 当 *表达式* 的值与某一个case后面的 *常量表达式*（全部由常量构成的表达式）的值相等时，就执行此case后面的 *语句* 。
- 若所有的 `case` 中的*常量表达式* 的值都没有与 *表达式* 的值匹配的，就执行 `default` 后面的语句。

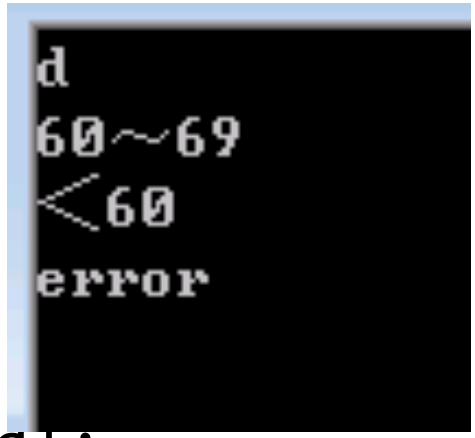
```cpp
#include <iostream>
using namespace std;
int main() {
    char grade = 'a';
    cin >> grade;
    switch (grade) {
        case 'a':
            cout << "85~100" << endl;
        case 'b':
            cout << "70~84" << endl;
        case 'c':
            cout << "60~69" << endl;
        case 'd':
            cout << "<60" << endl;
        default:
            cout << "error" << endl;
    }
    return 0;
}
```



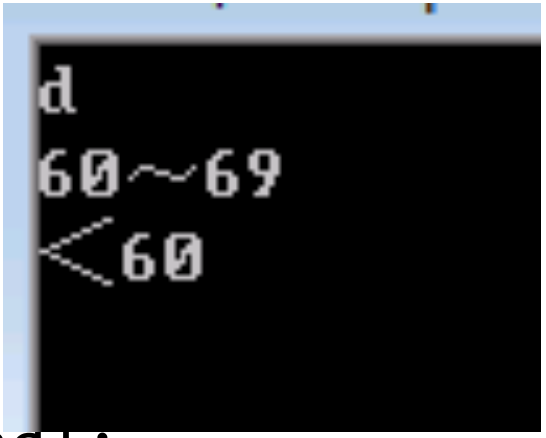
```cpp
#include <iostream>
using namespace std;
int main() {
    char grade = 'a';
    cin >> grade;
    switch (grade) {
        case 'a':
            cout << "85~100" << endl;
            break;
        case 'b':
            cout << "70~84" << endl;
            break;
        case 'c':
            cout << "60~69" << endl;
            break;
        case 'd':
            cout << "<60" << endl;
            break;
        default:
            cout << "error" << endl;
    }
    return 0;
}
```



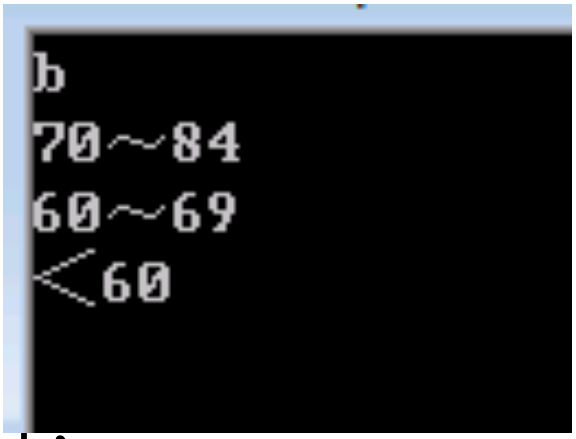
```cpp
#include <iostream>
using namespace std;
int main() {
    char grade = 'a';
    cin >> grade;
    switch (grade) {
        case 'a':
            cout << "85~100" << endl;
        case 'b':
            cout << "70~84" << endl;
        case 'c':
        case 'd':
        case 'e':
        case 'f':
            cout << "60~69" << endl;
        case 'g':
            cout << "<60" << endl;
        default:
            cout << "error" << endl;
    }
    return 0;
}
```



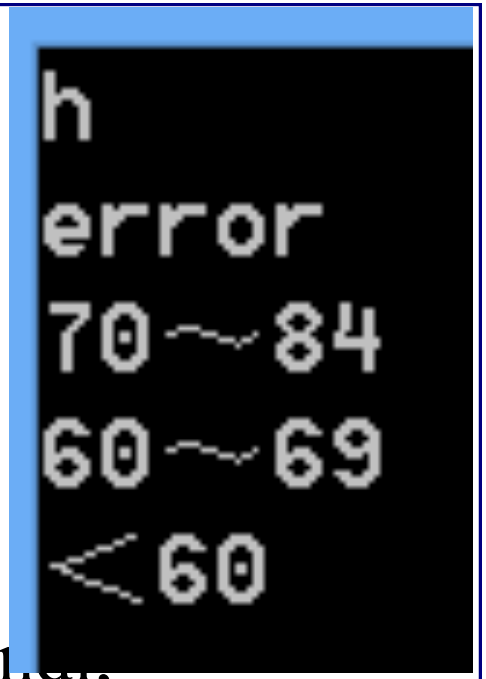
```cpp
#include <iostream>
using namespace std;
int main() {
    char grade = 'a';
    cin >> grade;
    switch (grade) {
        case 'a':
            cout << "85~100" << endl;
        case 'b':
            cout << "70~84" << endl;
        case 'c':
        case 'd':
        case 'e':
        case 'f':
            cout << "60~69" << endl;
        case 'g':
            cout << "<60" << endl;
            break;
        default:
            cout << "error" << endl;
    }
    return 0;
}
```



```cpp
#include <iostream>
using namespace std;
int main() {
    char grade = 'a';
    cin >> grade;
    switch (grade) {
        case 'a':
            cout << "85~100" << endl;
        default:
            cout << "error" << endl;
        case 'b':
            cout << "70~84" << endl;
        case 'c':
        case 'd':
        case 'e':
        case 'f':
            cout << "60~69" << endl;
        case 'g':
            cout << "<60" << endl;
    }
    return 0;
}
```



```
#include<iostream>
using namespace std;
int main()
{
        char grade = 'a';
        cin >> grade;
        switch (grade)
        {
                case 'a': cout << "85～100" << endl;
                default: cout << "error" << endl;
                case 'b': cout << "70～84" << endl;
                case 'c':
                case 'd':
                case 'e':
                case 'f': cout << "60～69" << endl;
                case 'g': cout << "＜60" << endl;
        }
        return 0;
}
```





思考与练习

- 问题：
  - 学校要求实行成绩等级制度。现在已经有同学们的百分制成绩,要求按照百分制成绩输出相应的等级成绩. 90-100为‘ A’, 80—90分为‘ B’,70—80分为‘ C’,60—70分为‘ D’,60分以下为‘ E’。
  - 最直接的办法：

    输入成绩；

    判定成绩是否： 90<=成绩<=100……

    判定成绩是否： 80<=成绩<90……

    ……
  - 如果用switch语句，如何解决?

再来看看 switch 语句

```
switch(表达式)
 {
  case  常量表达式1: 语句1;
  case  常量表达式2: 语句2;
    … ;
  case  常量表达式n: 语句n;
  default: 语句n＋1;
  }
```

- 90-100为’A’, 80—90分为’B’,70—80分为’C’,60—70分为’D’,60分以下为’E’。

```
#include<iostream>
using namespace std;
int main()
{
        int score, num;
        cout << "please give the score" << endl;
        cin >> score;
        num = score / 10;
        switch (num)
        {
                case 10: cout << "A" << endl;
                case  9: cout << "A" << endl;  // 也是A
                case  8: cout << "B" << endl;
                case  7: cout << "C" << endl;
                case  6: cout << "D" << endl;
                default: cout << "E" << endl;
        }
        return 0;
}
```

```
#include<iostream>
using namespace std;
int main()
{
        int score, num;
        cout << "please give the score" << endl;
        cin >> score;
        num = score / 10;
        switch (num)
        {
                case 10: cout << "A" << endl; break;
                case  9: cout << "A" << endl;  break;
                case  8: cout << "B" << endl;  break;
                case  7: cout << "C" << endl;  break;
                case  6: cout << "D" << endl;  break;
                default: cout << "E" << endl;
        }
        return 0;
}
```



再谈 循环结构

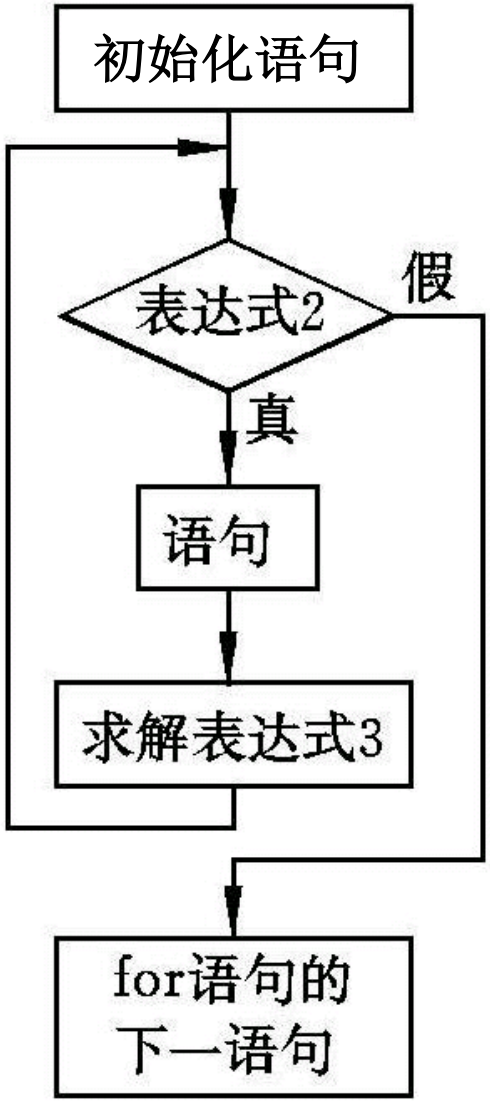
- C程序中的循环结构：
  - 用for语句构成循环。
  - 用while语句构成循环；
  - 用do…while语句构成循环；
  - 用goto语句和if语句构成循环；
- 循环中止或跳出语句：
  - 用continue语句结束本次循环；
  - 用 break 语句 跳出 本层循环；

for 语句

- for语句

```
for (初始化语句; 表达式2; 表达式3)
    语句
```

```
for (i = 0; i < 10; i++) {
    a = a * a;
}
```

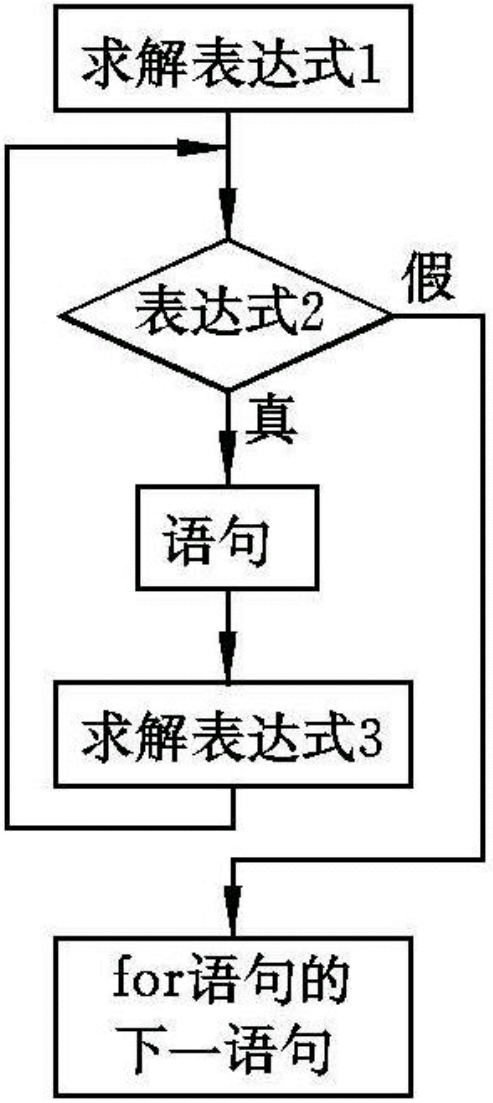


for 语 句

- for语句

```
for (表达式1; 表达式2; 表达式3)
        语句;
```

```
for ( i = 0; i < 10; i++ )
{
        a = a*a ;
}
```



for 语句

■ 一样的

```
for ( i = 0; i < 10; i++ )
 {
        a = a*a ;
 }
```

```
for ( i = 0; i < 10; ++i )
 {
        a = a*a ;
 }
```



for (i = 0; i < 10; i++)　和 for (i = 0; i < 10; ++i)　区别

已完成思考

在 `for` 循环的增量表达式中，`i++` 和 `++i` 的**实际效果完全相同**，都是让变量 `i` 的值每次增加 1，最终循环都会执行 10 次（`i` 从 0 到 9）。

核心区别：`i++` 与 `++i` 的本质差异

两者的区别体现在**表达式的返回值**上：

- `i++` 是**后置递增**：先返回 `i` 当前的值，再将 `i` 加 1。

  例如：`int a = i++;` 等价于 `int a = i; i = i + 1;`
- `++i` 是**前置递增**：先将 `i` 加 1，再返回 `i` 新的值。

  例如：`int a = ++i;` 等价于 `i = i + 1; int a = i;`

为什么在 for 循环中没区别？

`for` 循环的语法是 `for(初始化; 条件判断; 增量操作)`，其中**增量操作的返回值不会被使用**，编译器只关心“`i` 是否被加 1”这个动作，而不关心表达式返回了什么值。

for 语 句

- for语句

```
#include <iostream>
using namespace std;
int main()
{
        int i, a = 2;
        for(i = 0; i < 10; i++)
        {
            a = a*a;
            cout << i << " " << a << endl;
        }
        return 0;
}
```

```
0 4
1 16
2 256
3 65536
4 0
5 0
6 0
7 0
8 0
9 0
```

2^16

for 语 句

- for语句

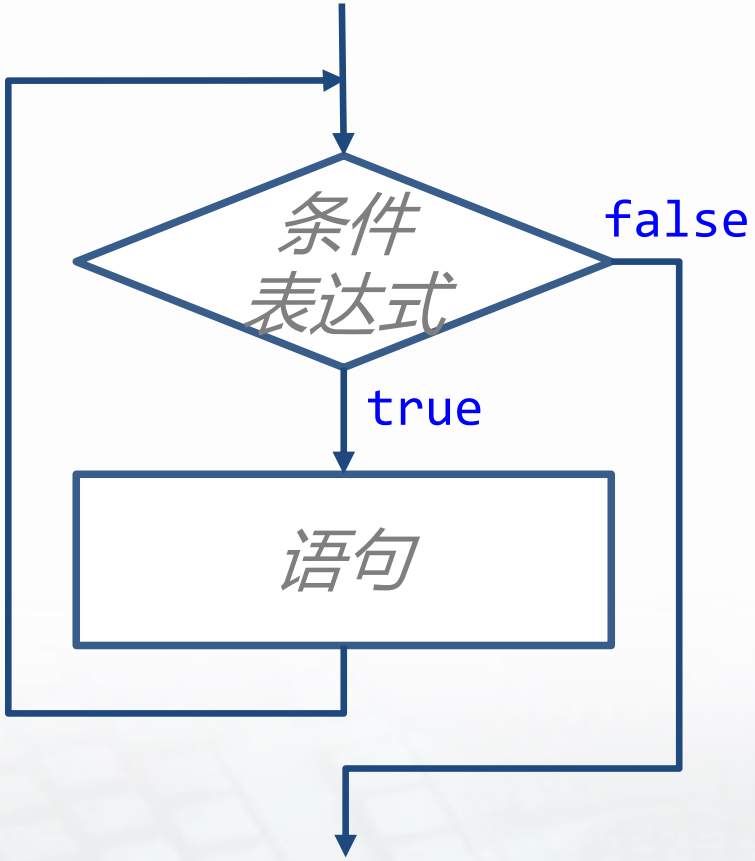
```
#include <iostream>
using namespace std;
int main()
{
        int a = 2;
        for(int i = 0; i < 10; i++)
        {
            a = a*a;
            cout << i << " " << a << endl;
        }
        return 0;
}
```

```
0 4
1 16
2 256
3 65536
4 0
5 0
6 0
7 0
8 0
9 0
```

2^16

while 语句

```
while (条件表达式)
    语句
```




```
#include <iostream>
using namespace std;
int main() {
    int i, sum = 0;
    i = 1;
    while (i <= 100) {
        sum = sum + i;
        i++;
    }
    cout << sum << endl;
    return 0;
}
```



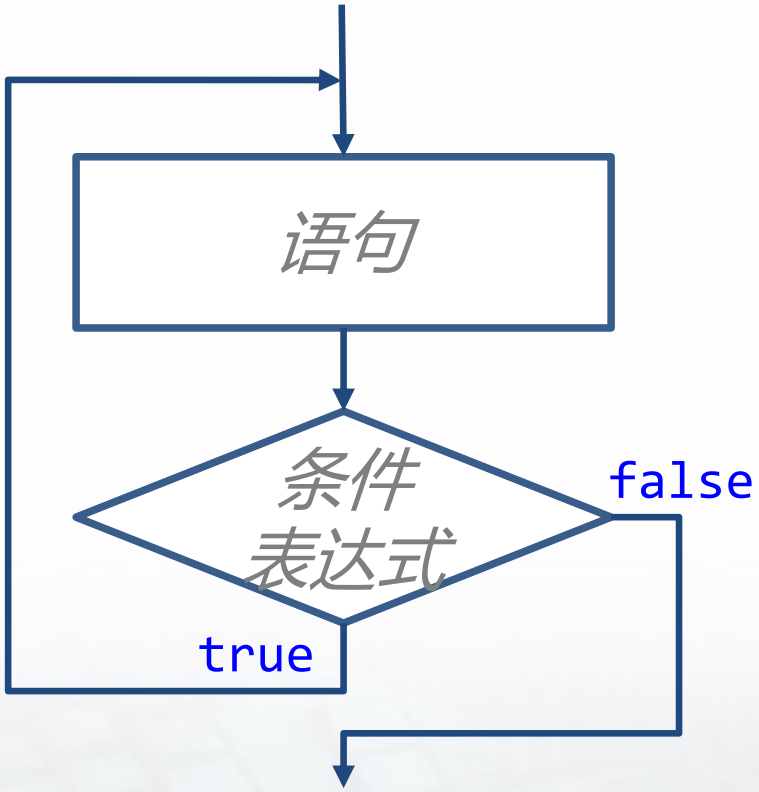
示 例

- 小红10岁,父亲33岁,问多少年之后,父亲的年龄是小红的二倍?

```
#include <iostream>
using namespace std;
int main() {
    int ageOfHong = 10, ageOfFather = 33, count = 0;
    while (2 * ageOfHong != ageOfFather) {
        ageOfHong++;
        ageOfFather++;
        count++;
    }
    cout << count;
    return 0;
}
```

do-while 语句

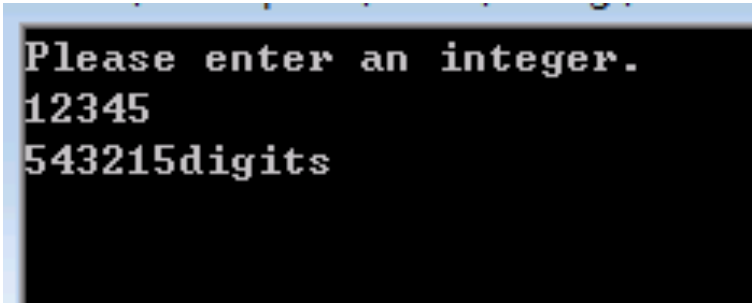
```
do
    语句
while (条件表达式);
```




```
#include <iostream>
using namespace std;
int main() {
    int i, sum = 0;
    i = 1;
    do {
        sum = sum + i;
        i++;
    } while (i <= 100);
    cout << sum << endl;
    return 0;
}
```

示 例

```
#include <iostream>
using namespace std;
int main() {
    int num;
    int count = 0;
    cout << "Please enter an integer." << endl;
    cin >> num;
    do {
        cout << num % 10;
        num = num / 10;
        count++;
    } while (num != 0);
    cout << count << "digits" << endl;
    return 0;
}
```





循环语句的嵌套

```
(1) while () {
        …
        while () {
          …
        }
    }
```

```
(2) do {
        …
        do {
          …
        } while ();
    } while ();
```

```
(3) for (;;) {
        for(;;) {
          …
        }
    }
```

```
(4) while () {
        …
        do {
          …
        } while();
        …
    }
```

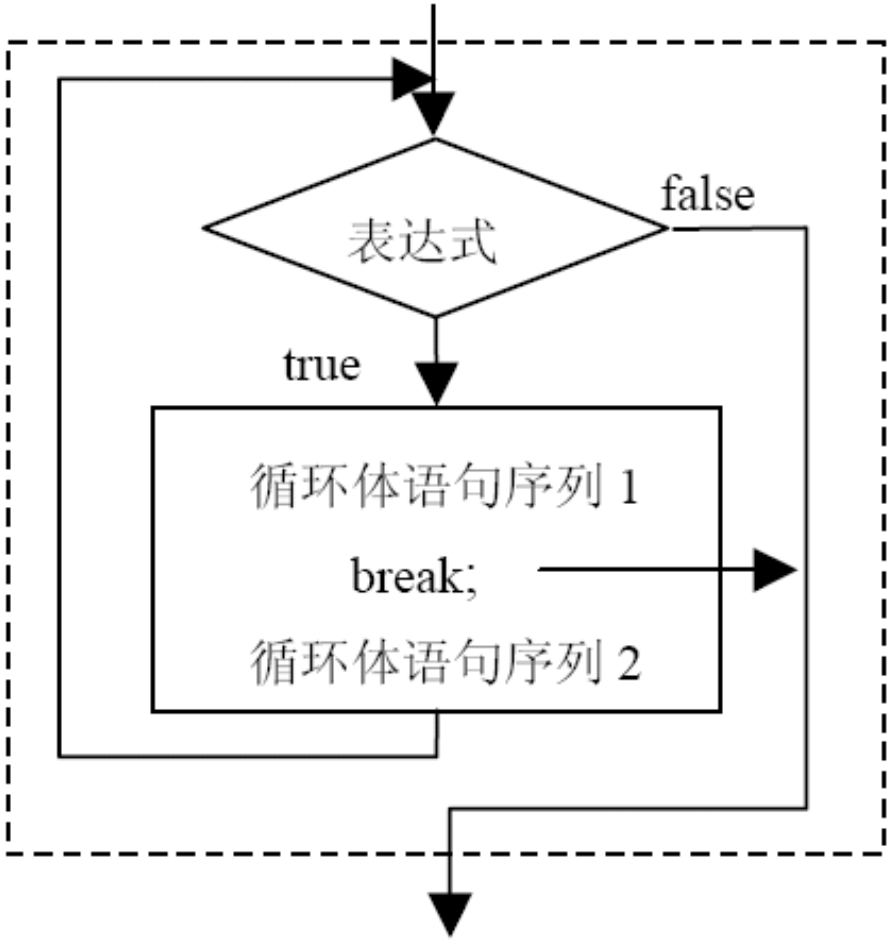
```
(5) for (;;) {
        …
        while () {
          …
        }
        …
    }
```

```
(6) do {
        …
        for (;;) {
          …
        }
    } while ();
```

转向控制语句

■ break 语句

- ◆ 在switch 语句、while 语句、do-while 语句、for 语句中使用;
- ◆ 以跳出switch语句或内层循环，继续执行逻辑上的下一条语句。



示例

```
#include <iostream>
using namespace std;
int main()
{
        int n = 0;
        for ( ; ; )
        {
            cin >> n;
            if (n == 0)
                break;
        }
        return 0;
}
```

```
#include <iostream>
using namespace std;
int main()
{
    int n = 0;
    while (true)
    {
        cin >> n;
        if (n == 0)
            break;
    }
    return 0;
}
```

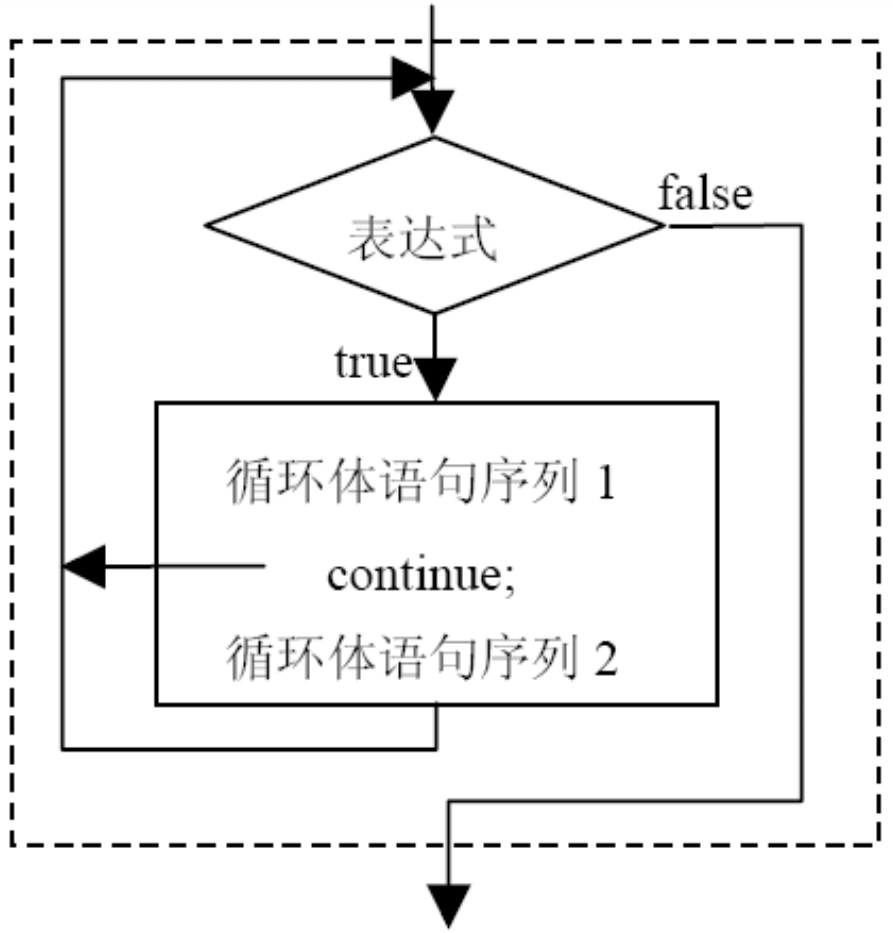
示例

```
while(true)
{
        for(…)
        {
            if (…)
                break;  // 跳出for循环
        }
        if (…)
            break;  // 跳出while循环
}
```

转向控制语句

■continue 语句

- ◆用于循环语句中;
- ◆结束本次循环，接着立即测试循环控制*表达式*，判断是否继续执行下一次循环。



示 例

输出1～100 之间所有数，除了能被3 或5 或7 整除的整数，每行显示10 个数。

```
#include <iostream>
using namespace std;
int main() {
    int n, counter = 0;
    for (n = 1; n <= 100; n++) {
        if (n % 3 == 0 || n % 5 == 0 || n % 7 == 0)
            continue;
        cout << n << '\t';
        counter++;
        if (counter % 10 == 0) cout << endl;
    }
    cout << endl;
    return 0;
}
```

```
1       2       4       8       11      13      16      17      19      22
23      26      29      31      32      34      37      38      41      43
44      46      47      52      53      58      59      61      62      64
67      68      71      73      74      76      79      82      83      86
88      89      92      94      97
```

直抒胸臆

——选择能够直接表达你的逻辑思路的控制语句！

goto 语句的例子

汇编语言

```
0x113e: mov  -0x8(%rbp),%eax
0x1141: add  %eax,-0x4(%rbp)
0x1144: addl $0x1,-0x8(%rbp)
0x1148: jmp  0x113e <main+0x19>
```

C++ 语言

```
#include <iostream>
using namespace std;
int main() {
    int i = 0, sum = 0;
    i = 1;
loop:
    sum = sum + i;
    i++;
    goto loop;
    cout << sum << endl;
    return 0;
}
```

早期的程序控制方法

- goto 语句
  - 无条件转向语句
  - 它的一般形式为

goto *语句标号*;

语句标号: *语句*

- 语句标号也是标识符，定名规则与变量名相同

```
end_of_while:;
// goto label - 1;   // 错误
// goto 123;         // 错误
goto end_of_while;   // 正确
```

空语句

goto 语句的例子

C++ 语言

```
#include <iostream>
using namespace std;
int main()
{
        int i = 0, sum = 0;
        i = 1;
loop:
        sum = sum + i;
        i++;
        if(i <= 100)
            goto loop;
        cout << sum << endl;
        return 0;
}
```

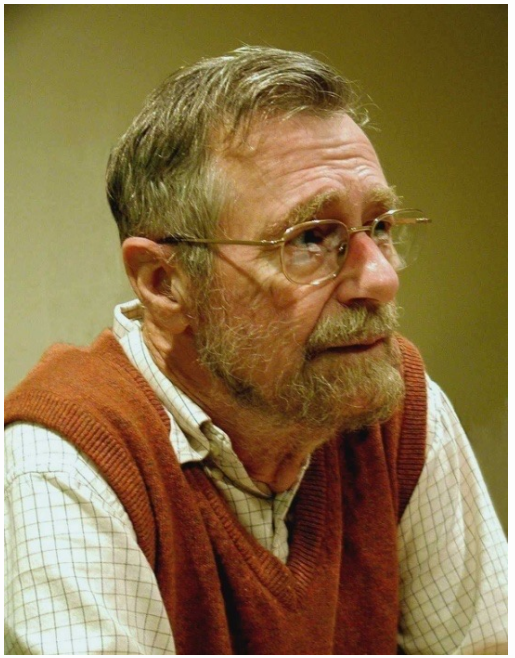
C++ 语言

```
#include <iostream>
using namespace std;
int main()
{
        int i = 0, sum = 0;
        i = 1;
        for(i=0; i <= 100; i++)
            sum = sum + i;
        cout << sum << endl;
        return 0;
}
```

```
5050
```

关于 goto 语句的讨论

- 著名的荷兰教授 E. W. Dijkstra
  - 1965年，IFIP（International Federation for Information Processing）会议上，Dijkstra提出“Go To语句可以从高级语言中取消”，“一个程序的质量与程序中所含的Go To语句的数量成反比”。
  - 但是，Dijkstra讲话的影响很小，当时人们正广泛地使用FORTRAN，而Go To语句则是FORTRAN的支柱。



Algol60设计实现者之一
GoTo有害论提出者
信号量理论提出者
最短路径算法提出者
THE操作系统设计者
程序正确性证明推动者



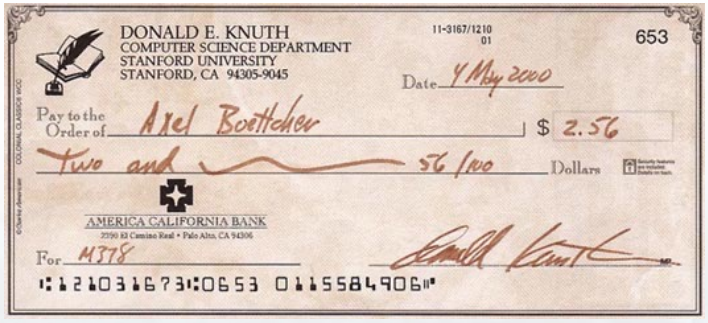
关于 goto 语句的讨论

- 60年代末至70年代，关于goto语句的争论非常激烈
- **正方**：从高级语言中去掉goto语句
  - 包含goto语句的程序难以阅读，难以查错；
  - 去掉goto语句后，可以直接从程序结构上反映程序的运行过程。使程序的结构清晰、便于阅读，便于查错，而且也有利于程序正确性的证明。
- **反方**：goto语句无害，应该保留
  - goto语句使用起来比较灵活，而且有些情形能够提高程序的效率。
  - 如果一味强调删除goto语句，有些情形反而会使程序过于复杂，增加一些不必要的计算量。

关于 goto 语句的讨论

- Donald E. Knuth（高德纳）
- 1974年，D.E.Knuth对于goto语句的争论作了全面的公正的评述：
  - 不加限制地使用goto语句，**特别是使用往回跳的goto语句**，会使程序的结构难于理解，这种情形应该尽量避免使用goto语句；
  - 为了提高程序的效率，同时又不破坏程序的良好结构，**有控制地使用一些goto语句是有必要的。**
- “有些情形，主张废除转向语句，有些情形我主张引进转向语句。”







总结

- if else
- switch case
- for loop
- while loop
- do … while
- break
- continue
- goto



想一想，有没有问题?